Part I - Airline On-Time Performance Data

by Jiri Valenta

Introduction

The dataset **"Airline On-Time Performance Data"** reports flights in the United States, including carriers, arrival and departure delays, and reasons for the delays, from 1987 to 2008. The data is collected by the Office of Airline Information, Bureau of Transportation Statistics (BTS). This notebook is to use exploratory visualisation techniques to analyse the dataset. In the first step let's have a look at the data and do some preliminary wrangling and analysis.

Table of Contents

- Preliminary Wrangling
- Univariable Exploration
- Bivariate Exploration
- Multivariate exploration
- Conclusions

Preliminary Wrangling

In [1]:

```
# import all packages and set plots to be embedded inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sb

%matplotlib inline
```

Loading the dataset

The original dataset is huge - unzipped it is 12GB in 22 CSV files. Although, the files are organized by years their sizes still cause issues when processing them repeatedly. As this project is mainly to demonstrate data visualization techniques I decided to extract just samples from each year's file and produce a merged CSV file of total size ~25MB. Once the code in this notebook is developed and tested the samples be increased or alternatively the whole dataset can be used.

> **Note:** The code below is inspired by the similar idea described at Udacity technical forum. The cell is intentionally left inactive as the merged file is already generated and saved into the workspace.

# extract a sample of 10k rows from each files and merge them together into the final CSV df_sample = pd.DataFrame() sample_years = np.arange(1987, 2009) # include all years from the source dataset sample_size = 10000 # how many sample rows from each year's source file for year in sample_years: file = 'DataExpo2009/' + str(year) + '.csv' print(file) df = pd.read_csv(file, encoding='unicode_escape').sample(sample_size) df_sample = df_sample.append(df, ignore_index=True) del df # this is to release memory as the files are large # save the final dataframe to a new source file df_sample.to_csv('sample_flights_2009.csv', index=False)

As an initial step in this notebook I read the file generated above and parse day, month and year columns into the new `Date` column of type Timestamp (leaving the time portion set as zero).

In [2]:

```
# read the extracted sample file into a new dataframe; parse dates
df = pd.read_csv('sample_flights_2009.csv', parse_dates={'Date':['Year','Month','DayofMonth']})
df.head()
```

```
/Users/jirivalenta/opt/anaconda3/lib/python3.8/site-packages/IPython/core/interactiveshell.py:3165: DtypeWarning: Columns (10,22) have mixed types.Specify dtype option on i
mport or set low_memory=False.
  has_raised = await self.run_ast_nodes(code_ast.body, cell_name,
```

Out[2]:

	Date	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum	ActualElapsedTime	...	TaxiIn	TaxiOut	Cancelled	CancellationCode	Diverted	CarrierDelay	WeatherD
0	1987-11-27	5	739.0	739	840.0	835	EA	280	NaN	61.0	...	NaN	NaN	0	NaN	0	NaN	
1	1987-12-20	7	650.0	644	925.0	919	UA	980	NaN	95.0	...	NaN	NaN	0	NaN	0	NaN	
2	1987-10-16	5	2025.0	2025	2128.0	2123	DL	785	NaN	63.0	...	NaN	NaN	0	NaN	0	NaN	
3	1987-12-11	5	1420.0	1420	1610.0	1610	WN	702	NaN	50.0	...	NaN	NaN	0	NaN	0	NaN	
4	1987-11-13	5	808.0	800	1644.0	1635	AA	44	NaN	336.0	...	NaN	NaN	0	NaN	0	NaN	

5 rows × 27 columns

In the next few cells I use Pandas displays to do a quick preliminary analysis of the data. I start with checking the datatypes used by Pandas when importing the file.

In [3]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 220000 entries, 0 to 219999
Data columns (total 27 columns):
 #   Column             Non-Null Count   Dtype         
---  ------             --------------   -----         
 0   Date               220000 non-null  datetime64[ns]
 1   DayOfWeek          220000 non-null  int64         
 2   DepTime            215976 non-null  float64       
 3   CRSDepTime         220000 non-null  int64         
 4   ArrTime            215472 non-null  float64       
 5   CRSArrTime         220000 non-null  int64         
 6   UniqueCarrier      220000 non-null  object        
 7   FlightNum          220000 non-null  int64         
 8   TailNum            139813 non-null  object        
 9   ActualElapsedTime  215467 non-null  float64       
 10  CRSElapsedTime     219954 non-null  float64       
 11  AirTime            136666 non-null  float64       
 12  ArrDelay           215467 non-null  float64       
 13  DepDelay           215976 non-null  float64       
 14  Origin             220000 non-null  object        
 15  Dest               220000 non-null  object        
 16  Distance           219557 non-null  float64       
 17  TaxiIn             139800 non-null  float64       
 18  TaxiOut            139820 non-null  float64       
 19  Cancelled          220000 non-null  int64         
 20  CancellationCode   1045 non-null    object        
 21  Diverted           220000 non-null  int64         
 22  CarrierDelay       48063 non-null   float64       
 23  WeatherDelay       48063 non-null   float64       
 24  NASDelay           48063 non-null   float64       
 25  SecurityDelay      48063 non-null   float64       
 26  LateAircraftDelay  48063 non-null   float64       
dtypes: datetime64[ns](1), float64(15), int64(6), object(5)
memory usage: 45.3+ MB
```

In [4]:

```
# display the number of cancelled flights
df_can = df.query('Cancelled == 1')
print("Number of cancelled flights:", df_can.shape[0])
df_can.head()
```

```
Number of cancelled flights: 4027
```

Out[4]:

	Date	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum	ActualElapsedTime	...	TaxiIn	TaxiOut	Cancelled	CancellationCode	Diverted	CarrierDelay	Weathe
86	1987-11-15	7	NaN	1145	NaN	1324	EA	783	NaN	NaN	...	NaN	NaN	1	NaN	0	NaN	
90	1987-12-28	1	NaN	2145	NaN	2229	UA	469	NaN	NaN	...	NaN	NaN	1	NaN	0	NaN	
175	1987-11-15	7	NaN	1711	NaN	1832	UA	403	NaN	NaN	...	NaN	NaN	1	NaN	0	NaN	
180	1987-12-02	3	NaN	140	NaN	235	HP	785	NaN	NaN	...	NaN	NaN	1	NaN	0	NaN	
197	1987-11-05	4	NaN	1345	NaN	1640	WN	537	NaN	NaN	...	NaN	NaN	1	NaN	0	NaN	

5 rows × 27 columns

There are a few columns that have categorical values so we can convert them accordingly. `DayOfWeek` can also be extracted from the newly created datetime `Date` however we can keep it as a separate column to ease our visualization. Let's convert `CancellationCode` categorical qualitative and `DayOfWeek` to categorical ordinal values.

In [5]:

```
# display all cancellation codes
reasons = df_can['CancellationCode'].value_counts()
reasons
```

Out[5]:

```
A    474
B    372
C    196
D      3
Name: CancellationCode, dtype: int64
```

I decided to convert cancellation codes into more descriptive categorical values according to the dataset description.

The definition of **NAS** code is: *Weather delays that could be reduced with corrective action by the airports or the Federal Aviation Administration.*

In [6]:

```
# convert CancellationCode into categorical variable
reasons = {'A':'Carrier', 'B':'Weather', 'C':'NAS', 'D':'Security'}
df['CancellationCode'] = df['CancellationCode'].map(reasons)
reason_classes = pd.api.types.CategoricalDtype(ordered=True, categories=reasons.values())
df['CancellationCode'] = df['CancellationCode'].astype(reason_classes)
df['CancellationCode'].value_counts()
```

Out[6]:

```
Carrier     474
Weather     372
NAS         196
Security      3
Name: CancellationCode, dtype: int64
```

In [7]:

```
# convert days of the week numbers into categorical variable
days_code = np.arange(1, 8)

# sorted days to be used throughout the notebook
days_of_week = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']
df['DayOfWeek'] = df['DayOfWeek'].map(dict(zip(days_code, days_of_week))).values
week_days = pd.api.types.CategoricalDtype(ordered=True, categories=days_of_week)
df['DayOfWeek'] = df['DayOfWeek'].astype(week_days)
```

In [8]:

```
df.head()
```

Out[8]:

	Date	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum	ActualElapsedTime	...	TaxiIn	TaxiOut	Cancelled	CancellationCode	Diverted	CarrierDelay	WeatherD
0	1987-11-27	Fri	739.0	739	840.0	835	EA	280	NaN	61.0	...	NaN	NaN	0	NaN	0	NaN	
1	1987-12-20	Sun	650.0	644	925.0	919	UA	980	NaN	95.0	...	NaN	NaN	0	NaN	0	NaN	
2	1987-10-16	Fri	2025.0	2025	2128.0	2123	DL	785	NaN	63.0	...	NaN	NaN	0	NaN	0	NaN	
3	1987-12-11	Fri	1420.0	1420	1610.0	1610	WN	702	NaN	50.0	...	NaN	NaN	0	NaN	0	NaN	
4	1987-11-13	Fri	808.0	800	1644.0	1635	AA	44	NaN	336.0	...	NaN	NaN	0	NaN	0	NaN	

5 rows × 27 columns

Let's have a closer look at some other columns.

`TailNum` is a kind of an aircraft registration number. The most of the values are missing and there is no way to recover them. It may be useful to link them with aircraft types to see if certain type is more involved in delays or cancellations but I don't have this information at hand so I won't be using the column. I can drop the column.

In [9]:

```
# display a few TailNum values
df['TailNum'].value_counts()
```

Out[9]:

```
UNKNOW    1040
0          523
äNKNOæ     342
000000      95
N523        74
          ... 
N336ä1       1
N648Aæ       1
N921L@       1
N322TZ       1
-N618D       1
Name: TailNum, Length: 11960, dtype: int64
```

In [10]:

```
# drop TailNum column
df.drop(columns='TailNum', inplace=True)
```

`Date` column was produced during parsing the source CSV file and as we have just one date available for each row I assume it is the date of departure - used in conjunction with the flight number to uniquely identify the flight. However we can't use it together with `ArrTime` as the date can be different. Also timezone has to be taken into account as all times used in our dataset are 'local' = relative to the airport location.

Anyway for the purpose of parsing them later we can convert them from 'float' to zero prefixed strings of 4 digits.

In [11]:

```
# display the number of the "next day arrivals"
sum(df['DepTime'] > df['ArrTime'])
```

Out[11]:

```
6268
```

In [12]:

```
# convert times from float to strings (4)
df['DepTime'] = df['DepTime'].astype('Int64').astype(str).str.zfill(4)
df['ArrTime'] = df['ArrTime'].astype('Int64').astype(str).str.zfill(4)
df.head()
```

Out[12]:

	Date	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	ActualElapsedTime	CRSElapsedTime	...	TaxiIn	TaxiOut	Cancelled	CancellationCode	Diverted	CarrierDelay	W
0	1987-11-27	Fri	0739	739	0840	835	EA	280	61.0	56.0	...	NaN	NaN	0	NaN	0	NaN	
1	1987-12-20	Sun	0650	644	0925	919	UA	980	95.0	95.0	...	NaN	NaN	0	NaN	0	NaN	
2	1987-10-16	Fri	2025	2025	2128	2123	DL	785	63.0	58.0	...	NaN	NaN	0	NaN	0	NaN	
3	1987-12-11	Fri	1420	1420	1610	1610	WN	702	50.0	50.0	...	NaN	NaN	0	NaN	0	NaN	
4	1987-11-13	Fri	0808	800	1644	1635	AA	44	336.0	335.0	...	NaN	NaN	0	NaN	0	NaN	

5 rows × 26 columns

For the departure times we can join them together with `Date` to produce complete timestamps. As mentioned above we can't do it easily for arrival times. Let's create a new `DepTS` column with a full (date & time) departure timestamp.

In [13]:

```
# join Date and DepTime into a new Timestamp column
df['DepTS'] = pd.to_datetime(df['Date'].dt.strftime('%Y-%m-%d') + df['DepTime'], format='%Y-%m-%d%H%M', errors='coerce')
df['DepTS'].head()
```

Out[13]:

```
0   1987-11-27 07:39:00
1   1987-12-20 06:50:00
2   1987-10-16 20:25:00
3   1987-12-11 14:20:00
4   1987-11-13 08:08:00
Name: DepTS, dtype: datetime64[ns]
```

Times Conversion

In order to be able to compare and calculate differences between times I convert all Times into timestamps. When only the time is used, the date part will be set as 1900-01-01 for all by default.

CRS-prefixed times are coming from "Customer Reservation System". These times are *scheduled* times - expected departures / arrivals. The other times (not CRS) are the actual times received from automated departure systems.

The elapsed times, delays, taxi and air times are in minutes and for our purpose we can convert them into integers to simplify the future presentation.

In [14]:

```
# convert all times into Timestamps
df['DepTime'] = pd.to_datetime(df['DepTime'], format='%H%M', errors='coerce')
df['ArrTime'] = pd.to_datetime(df['ArrTime'], format='%H%M', errors='coerce')
df['CRSDepTime'] = pd.to_datetime(df['CRSDepTime'].astype(str), format='%H%M', errors='coerce')
df['CRSArrTime'] = pd.to_datetime(df['CRSArrTime'].astype(str), format='%H%M', errors='coerce')

# convert these columns to int
times_to_int = ['ActualElapsedTime','CRSElapsedTime','TaxiOut','TaxiIn','ArrDelay','DepDelay','AirTime',\
            'CarrierDelay','WeatherDelay','NASDelay','SecurityDelay','LateAircraftDelay']
for col in times_to_int:
    df[col] = df[col].astype('Int64')

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 220000 entries, 0 to 219999
Data columns (total 27 columns):
 #   Column             Non-Null Count   Dtype         
---  ------             --------------   -----         
 0   Date               220000 non-null  datetime64[ns]
 1   DayOfWeek          220000 non-null  category      
 2   DepTime            215961 non-null  datetime64[ns]
 3   CRSDepTime         218733 non-null  datetime64[ns]
 4   ArrTime            215387 non-null  datetime64[ns]
 5   CRSArrTime         218486 non-null  datetime64[ns]
 6   UniqueCarrier      220000 non-null  object        
 7   FlightNum          220000 non-null  int64         
 8   ActualElapsedTime  215467 non-null  Int64         
 9   CRSElapsedTime     219954 non-null  Int64         
 10  AirTime            136666 non-null  Int64         
 11  ArrDelay           215467 non-null  Int64         
 12  DepDelay           215976 non-null  Int64         
 13  Origin             220000 non-null  object        
 14  Dest               220000 non-null  object        
 15  Distance           219557 non-null  float64       
 16  TaxiIn             139800 non-null  Int64         
 17  TaxiOut            139820 non-null  Int64         
 18  Cancelled          220000 non-null  int64         
 19  CancellationCode   1045 non-null    category      
 20  Diverted           220000 non-null  int64         
 21  CarrierDelay       48063 non-null   Int64         
 22  WeatherDelay       48063 non-null   Int64         
 23  NASDelay           48063 non-null   Int64         
 24  SecurityDelay      48063 non-null   Int64         
 25  LateAircraftDelay  48063 non-null   Int64         
 26  DepTS              215961 non-null  datetime64[ns]
dtypes: Int64(12), category(2), datetime64[ns](6), float64(1), int64(3), object(3)
memory usage: 44.9+ MB
```

In [15]:

```
# display overall statistical information about the dataset
df.describe()
```

Out[15]:

	FlightNum	ActualElapsedTime	CRSElapsedTime	AirTime	ArrDelay	DepDelay	Distance	TaxiIn	TaxiOut	Cancelled	Diverted	CarrierDelay	Wea
count	220000.000000	215467.000000	219954.000000	136666.000000	215467.000000	215976.000000	219557.000000	139800.000000	139820.000000	220000.000000	220000.000000	48063.000000	4806
mean	1275.241818	118.742490	119.489493	102.650169	7.096804	8.051075	693.684925	6.359864	15.140352	0.018305	0.002300	3.694547	
std	1314.134819	68.126454	67.534852	69.992430	30.218079	27.747210	546.688821	19.923082	10.879200	0.134051	0.047903	20.579165	
min	1.000000	8.000000	15.000000	-1411.000000	-143.000000	-501.000000	11.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	432.000000	69.000000	70.000000	54.000000	-7.000000	-2.000000	304.000000	4.000000	10.000000	0.000000	0.000000	0.000000	
50%	884.000000	100.000000	100.000000	84.000000	0.000000	0.000000	533.000000	5.000000	13.000000	0.000000	0.000000	0.000000	
75%	1634.000000	150.000000	150.000000	133.000000	11.000000	6.000000	929.000000	7.000000	18.000000	0.000000	0.000000	0.000000	
max	9584.000000	1535.000000	660.000000	1495.000000	1239.000000	1435.000000	4962.000000	1455.000000	766.000000	1.000000	1.000000	1168.000000	89

Obviously some values may be incorrect, like max values for delays and negative airtime values. I may need to filter them out as outliers in the following visualizations. All the negative or missing airtimes can be set to 0 assuming we will use only values >0 for visualizations.

In [16]:

```
# display the numbers of occasions when the airtime is negative
print(df[df['AirTime'] < 0]['AirTime'].value_counts())

# set NaN or negative values to 0
df['AirTime'] = df['AirTime'].fillna(0).apply(lambda x: x if x > 0 else np.NaN)
```

```
-1381    2
-1387    2
-1354    1
-715     1
-12      1
-1357    1
-36      1
-656     1
-1361    1
-1364    1
-1367    1
-1382    1
-1371    1
-1372    1
-356     1
-1318    1
-1388    1
-1390    1
-1392    1
-1356    1
-1330    1
-1078    1
-1335    1
-1359    1
-1339    1
-1411    1
-1343    1
Name: AirTime, dtype: Int64
```

`TaxiIn` and `TaxiOut` values are also sometimes out of expected ranges, like the max taxi time of 1455 mins. These values are usually automatically collected by departure systems using radio communication from aircrafts. Sometimes the readings can be unreliable, missing or manually overridden.

In [17]:

```
df.query('TaxiIn == 1455.0')
```

Out[17]:

	Date	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	ActualElapsedTime	CRSElapsedTime	...	TaxiOut	Cancelled	CancellationCode	Diverted	CarrierDelay	We
171070	2004-06-02	Wed	1900-01-01 22:49:00	1900-01-01 22:49:00	NaT	1900-01-01 02:02:00	EV	4564	96	93	...	12	0	NaN	0	0	

1 rows × 27 columns

In [18]:

```
# final check of the dataframe
df.head()
```

Out[18]:

	Date	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	ActualElapsedTime	CRSElapsedTime	...	TaxiOut	Cancelled	CancellationCode	Diverted	CarrierDelay	Weather
0	1987-11-27	Fri	1900-01-01 07:39:00	1900-01-01 07:39:00	1900-01-01 08:40:00	1900-01-01 08:35:00	EA	280	61	56	...	<NA>	0	NaN	0	<NA>	
1	1987-12-20	Sun	1900-01-01 06:50:00	1900-01-01 06:44:00	1900-01-01 09:25:00	1900-01-01 09:19:00	UA	980	95	95	...	<NA>	0	NaN	0	<NA>	
2	1987-10-16	Fri	1900-01-01 20:25:00	1900-01-01 20:25:00	1900-01-01 21:28:00	1900-01-01 21:23:00	DL	785	63	58	...	<NA>	0	NaN	0	<NA>	
3	1987-12-11	Fri	1900-01-01 14:20:00	1900-01-01 14:20:00	1900-01-01 16:10:00	1900-01-01 16:10:00	WN	702	50	50	...	<NA>	0	NaN	0	<NA>	
4	1987-11-13	Fri	1900-01-01 08:08:00	1900-01-01 08:00:00	1900-01-01 16:44:00	1900-01-01 16:35:00	AA	44	336	335	...	<NA>	0	NaN	0	<NA>	

5 rows × 27 columns

This was the final step of the preliminary data wrangling process. Obviously, there can be much more cleaning done on the dataset. Also, many columns are redundant, like CRS times which were used to calculate delays.

Let's save the cleaned dataframe into a new CSV file for the reference.

In [19]:

```
# save the cleaned dataframe to CSV file
df.to_csv('sample_flights_2009_clean.csv', index=False)
df.shape
```

Out[19]:

```
(220000, 27)
```

What is the structure of your dataset?

The dataset was generated by merging the samples from 22 CSV files and is organized as a single table. The rows represent flights with all the data captured between departure and arrival. Each row is accompanied with the time and date of the departure. There are also columns with delays in minutes under separate categories. As there can be multiple delays categories used for one flight the columns are kept separately as in original file.

What is/are the main feature(s) of interest in your dataset?

The dataset allows investigation of many different characteristics and relationships. I would like to focus on departure and arrival delays and also to see whether there are any trends within the time range captured in the dataset. I may also try to look at the performance of the different carriers and the distribution of reasons for the cancellations.

What features in the dataset do you think will help support your investigation into your feature(s) of interest?

I will use `DateTS` column for any plots with timeseries. To identify the carriers I can use `UniqueCarrier` column as all the flights in the dataset seem to be operated (not marketed) by the specified carriers (flight numbers are either 3 digits or 4 but operated by the airline according to SkyScanner). I will also use most of the 'elapsed time in minutes' columns, like delays, and also airport codes and cancellation reason codes.

Univariate Exploration

In this section I look at the distribution of the main variables of my interest separately using bar charts.

Question 1:

What is an overall distribution of departure and arrival delays in the dataset?

First, let's see whether we need to deal with any missing values in our histogram plots. My plan is not to delete the rows entirely but rather consider the missing values for each plot and handle them separately as needed.

In [20]:

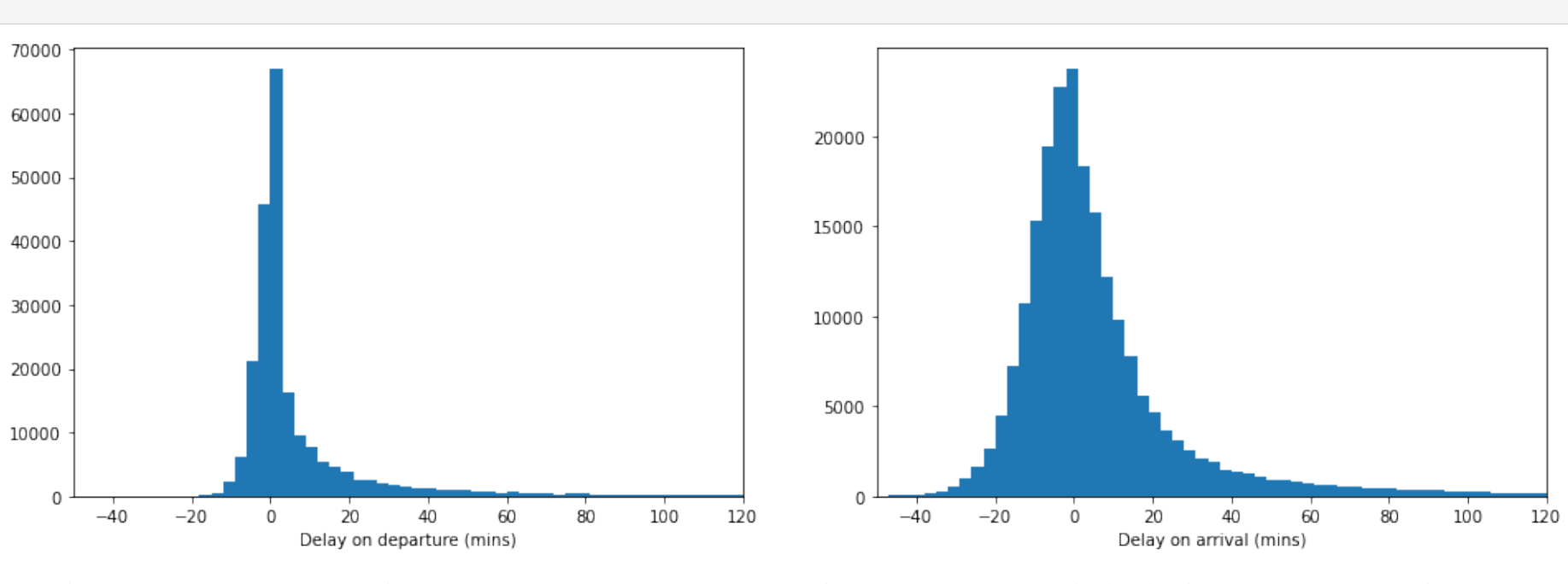
```
sum(df['DepDelay'].isnull()), sum(df['ArrDelay'].isnull())
```

Out[20]:

```
(4024, 4533)
```

In [21]:

```
# plot the distribution of departure and arrival delays (can be negative)
binsize = 3
plt.figure(figsize = [16, 5])

plt.subplot(1, 2, 1)
bins = np.arange(df['DepDelay'].min(), df['DepDelay'].max() + binsize, binsize)
plt.hist(data=df[df['DepDelay'].isnull() == False], x='DepDelay', bins=bins);
plt.xlim(-50, 120);
plt.xlabel('Delay on departure (mins)');

plt.subplot(1, 2, 2)
bins = np.arange(df['ArrDelay'].min(), df['ArrDelay'].max() + binsize, binsize)
plt.hist(data=df[df['ArrDelay'].isnull() == False], x='ArrDelay', bins=bins);
plt.xlim(-50, 120);   # use the same range as for departures
plt.xlabel('Delay on arrival (mins)');
```



Both histograms have the same x-axis ranges so that they can be compared visually. The departure distribution is narrower and more right skewed. As expected the arrival delays have higher deviation from zero as they already 'absorbed' departure delays plus any addition delays incurred during the flight. Surprisingly, there is a lot of flights that departed or arrived earlier than scheduled. It would be good to analyse this feature again using the bivariate exploration.

Question 2:

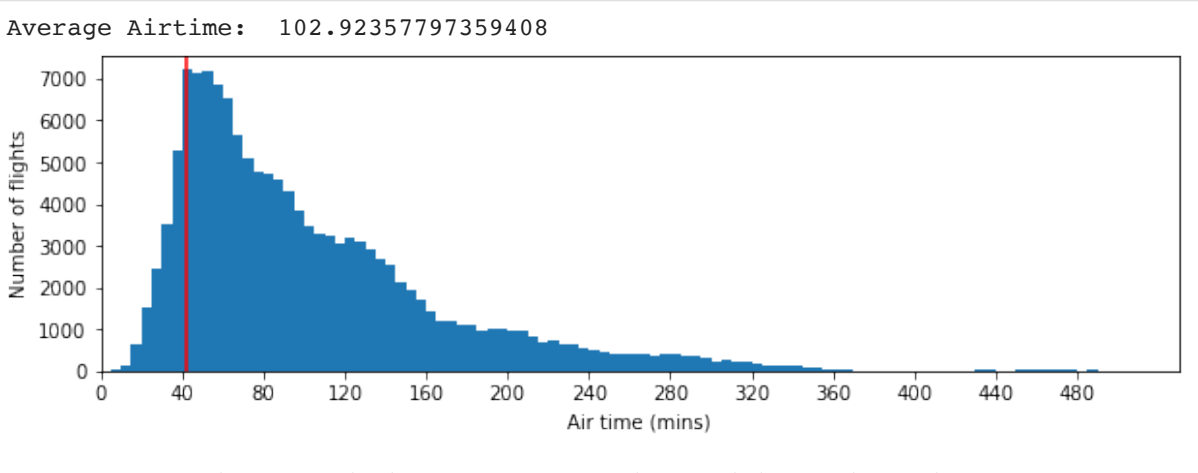
What is a typical airtime distribution?

To get an idea of the typical flight airtime let's plot a simple histogram again showing the distribution.

In [22]:

```
# plot Air time distribution
binsize = 5
plt.figure(figsize = [10, 3])
bins = np.arange(0, df['AirTime'].max() + binsize, binsize)
x, y, _ = plt.hist(data=df[df['AirTime'] > 0], x='AirTime', bins=bins);
xmax = y[np.where(x == x.max())]   # get the max value on x-axis
plt.axvline(x=xmax+binsize/2, color='red')   # mark the max value in the plot
plt.xlim(0, 530);
plt.xlabel('Air time (mins)');
plt.ylabel('Number of flights')
plt.xticks(np.arange(0, 500, 40));

# print an avarage airtime
print("Average Airtime: ", df[df['AirTime'] != np.NaN]['AirTime'].mean())
```

```
Average Airtime:  102.92357797359408
```



The most of the flights are distributed around 40 minutes airtime. This confirms that the dataset contains a high number of relatively short domestic flights.

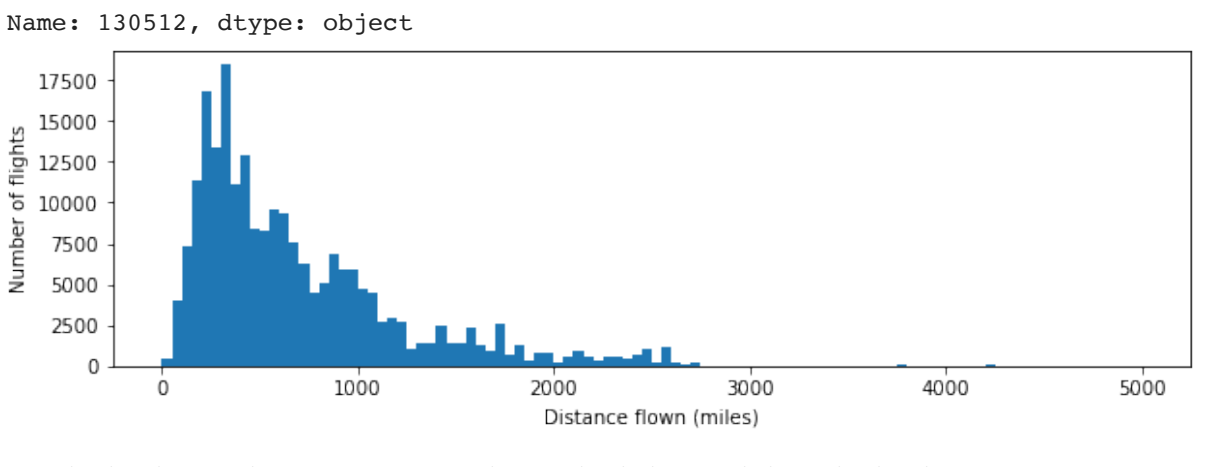
Question 3:

What is a distribution of the distance flown? Is it corresponding to the airtimes?

In [23]:

```
# Distance between airports
binsize = 50
plt.figure(figsize = [10, 3])
bins = np.arange(0, df['Distance'].max() + binsize, binsize)
plt.hist(data=df, x='Distance', bins=bins);
plt.xlabel('Distance flown (miles)');
plt.ylabel('Number of flights')

print("Max airports distance (miles):", df.Distance.max())
print(df.query('Distance == Distance.max()')[['Origin','Dest']].iloc[0])
```

```
Max airports distance (miles): 4962.0
Origin    EWR
Dest      HNL
Name: 130512, dtype: object
```



The distribution of distances between airports is similar to airtime distribution as expected. The maximum distance flown is between Honolulu (HA) and New York (NJ).

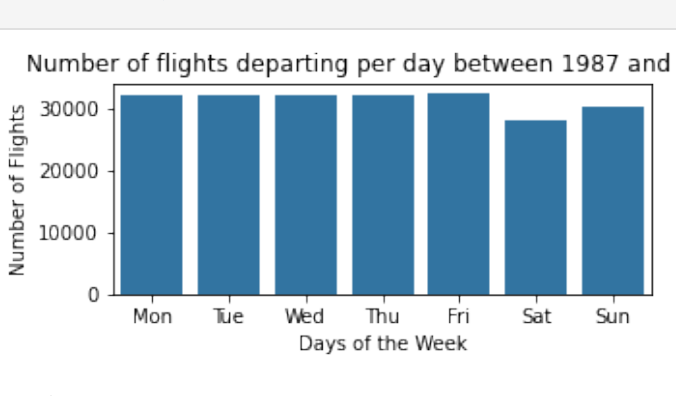
Question 4:

How many flights are departing each day of the week?

Let's use seaborn countplot to plot a barchart showing each day of the week.

In [24]:

```
# plot flight counts for each day of the week
plt.figure(figsize = [5, 2])
default_color = sb.color_palette()[0]
ax = sb.countplot(data=df, x='DayOfWeek', color=default_color, order=days_of_week);
plt.xlabel('Days of the Week');
plt.ylabel('Number of Flights');
plt.title('Number of flights departing per day between 1987 and 2008');
```



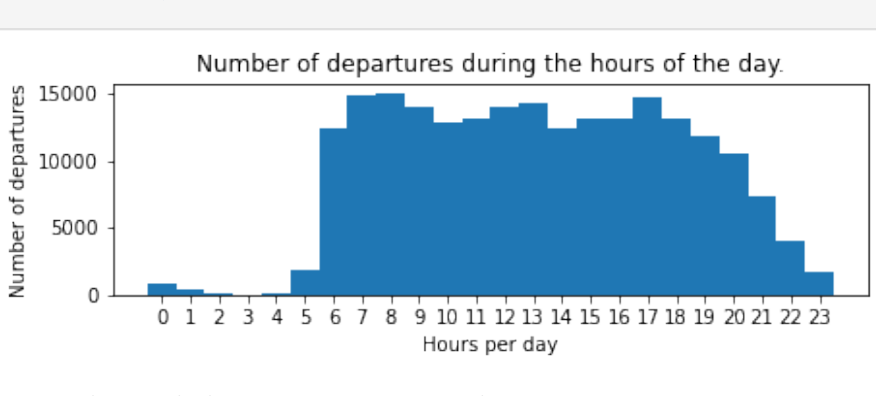
Obviously, Saturday and Sunday are the least busy days of the week in terms of departures.

Question 5:

What are the busiest departure hours per day?

In [25]:

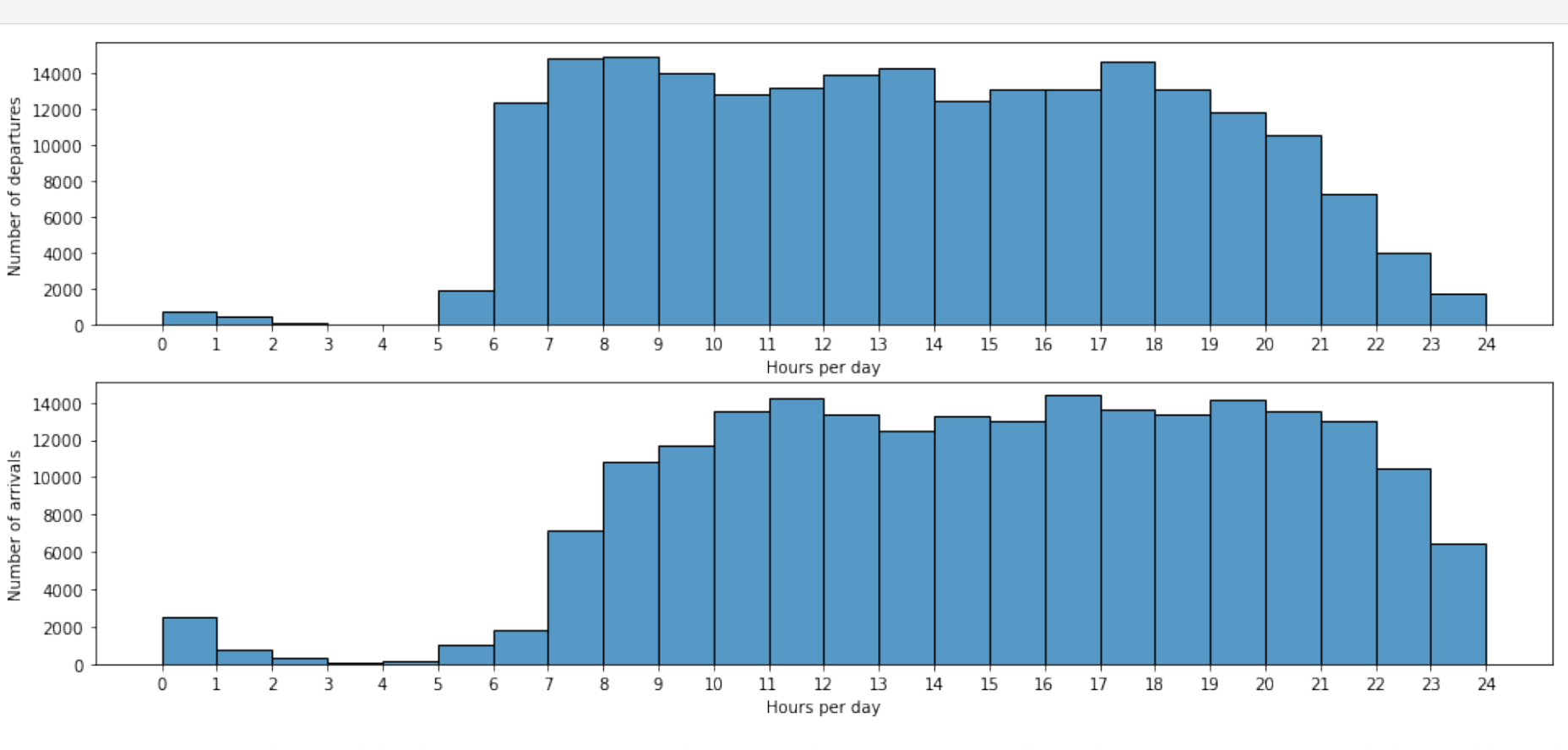
```
# plot busy hours for departures
plt.figure(figsize = [7, 2])
df_hrs = df.groupby(df.DepTime.dt.hour)['Date'].count()
plt.bar(data=df_hrs, x=df_hrs.index, height=df_hrs.values, width=1.0);
plt.xticks(np.arange(0,24));
plt.ylabel('Number of departures')
plt.xlabel('Hours per day');
plt.title('Number of departures during the hours of the day.');
```



The main operiations start after 6am with peaks around 8am, 1pm and 5pm. Since arrival hours are local to the destination airport, let's compare both departure and arrival distributions throughout the day. I use seaborn histplot for the comparison.

In [26]:

```
# plot histogram of flight departure/arrival frequencies during day's hours
bin_edges = np.arange(0, 25)
xticks = np.arange(0, 25, 1)
plt.figure(figsize = [16, 7])
default_color = sb.color_palette()[0]

# departures
plt.subplot(2, 1, 1)
sb.histplot(df.DepTS.dt.hour, color=default_color, bins=bin_edges)
plt.xticks(xticks)
plt.xlabel('Hours per day')
plt.ylabel('Number of departures');

# arrivals
plt.subplot(2, 1, 2)
sb.histplot(df.ArrTime.dt.hour, color=default_color, bins=bin_edges)
plt.xticks(xticks)
plt.xlabel('Hours per day')
plt.ylabel('Number of arrivals');
```



As expected the frequencies are similar just the departures are slightly more right-skewed and arrivals slightly left-skewed. There is little activity during night hours most probably due to airport regulations for domestic flights.

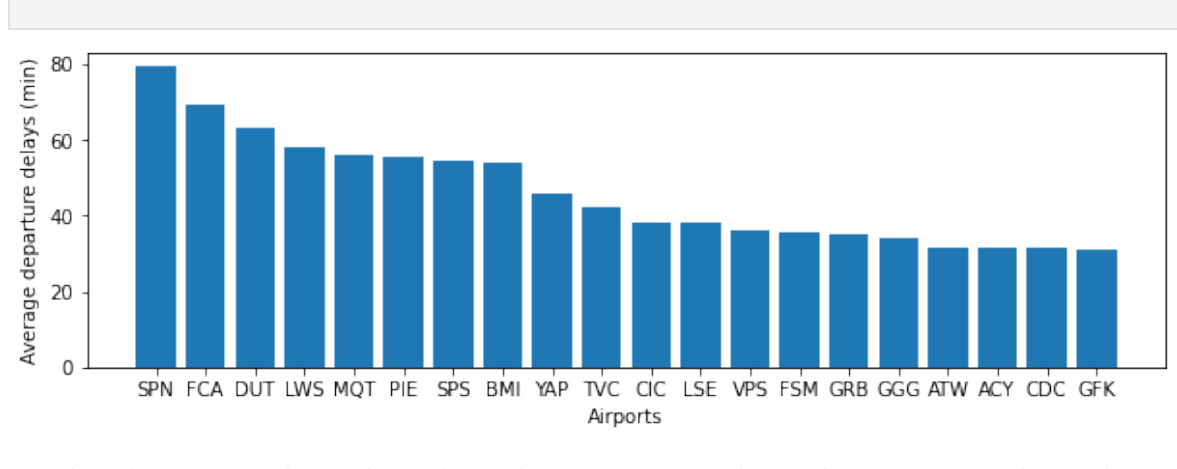
Question 6:

What are the airports with the highest average departure delays?

I use barchart to display just a first few airport that experience the highest frequency of departure delays.

In [27]:

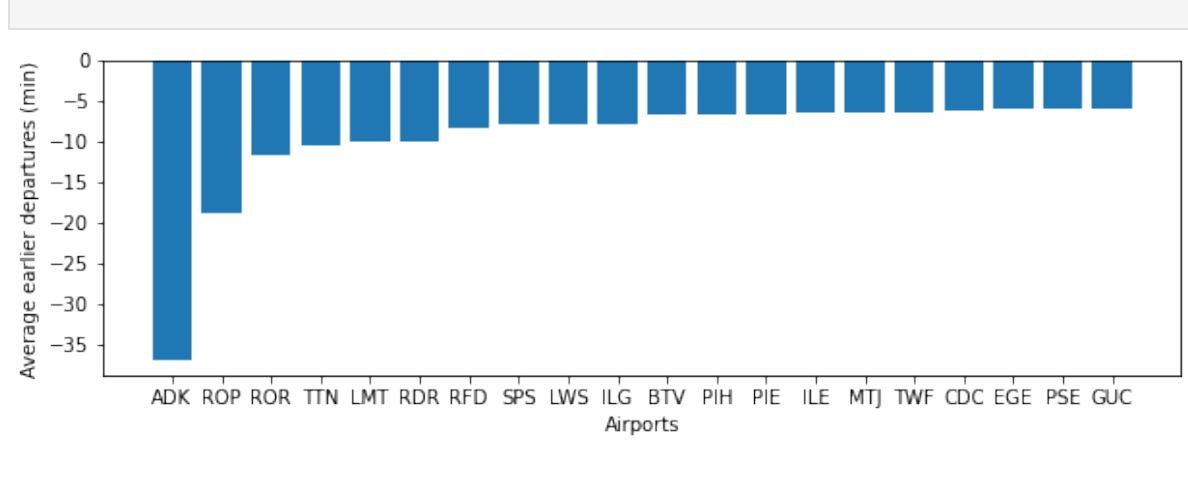
```
# airports with the highest departure delays - don't include early departures into the mean
locs_dep_del = df[df['DepDelay'] >= 0].groupby('Origin')['DepDelay'].mean().sort_values(ascending=False).head(20)
plt.figure(figsize = [10, 3])
plt.bar(x=locs_dep_del.index, height=locs_dep_del.values);
plt.xlabel('Airports')
plt.ylabel('Average departure delays (min)');
```



SPN is Saipan International airport in Mariana Islands; FCA is Glacier Park International Airport. Both are relatively small regional airports which means they are not frequently represented in our dataset and thus this visualisation is not very accurate. The same applies to the airports on the other side of the spectrum - with earlies departues.

In [28]:

```
# plot earlier than scheduled departures
locs_dep_ear = df[df['DepDelay'] <= 0].groupby('Origin')['DepDelay'].mean().sort_values(ascending=True).head(20)
plt.figure(figsize = [10, 3])
plt.bar(x=locs_dep_ear.index, height=locs_dep_ear.values);
plt.xlabel('Airports')
plt.ylabel('Average earlier departures (min)');
```



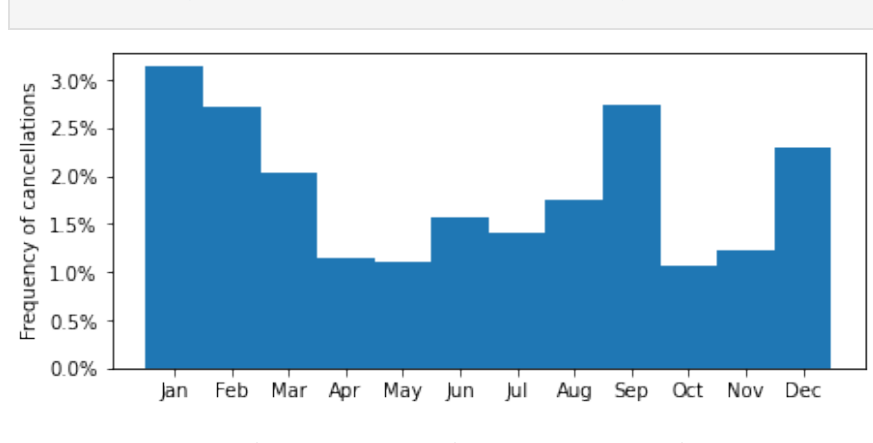
Question 7:

What month of the year is more vulnerable to cancellations?

The following bar plot shows the monthly frequency of cancellations throughout the year.

In [29]:

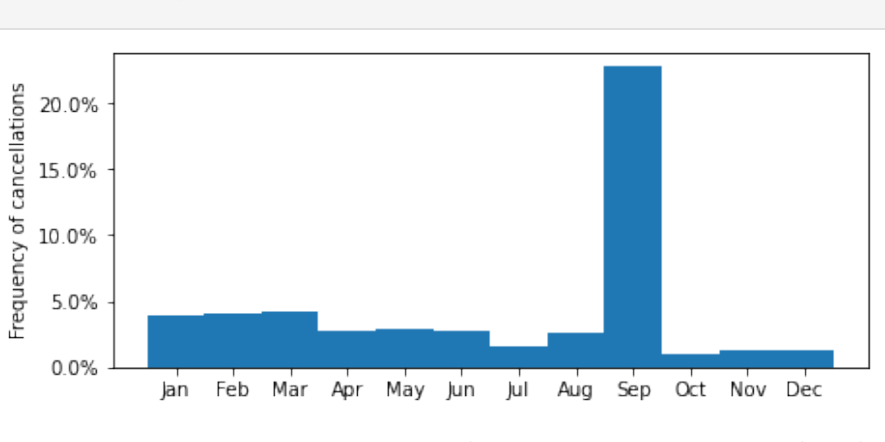
```
# prepare a new dataframe for the bar plot
month_labels = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
fig, ax = plt.subplots(1, 1, figsize=(7,3))
freq_cancel = df.groupby(df.Date.dt.month)['Cancelled'].mean()

# plot frequency of cancellations per month
ax.bar(x=freq_cancel.index, height=freq_cancel.values, width=1.0);
ax.set_xticks(np.arange(1,13))
ax.set_xticklabels(month_labels);
yticks = mtick.PercentFormatter(xmax=1, decimals=1)
ax.yaxis.set_major_formatter(yticks)
plt.ylabel('Frequency of cancellations');
```



It can be expected, in northern hemisphere, that the high occurrence of cancellations is during the winter months however the peak in September is quite unusual. Let's repeat the same barchart only for the year 2001 (9/11 attacks) to see whether this could have had an impact to the overal statistics (1987 to 2008).

In [30]:

```
# frequency of cancellations per month in 2001 (9/11)
month_labels = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
fig, ax = plt.subplots(1, 1, figsize=(7,3))
freq_cancel = df[df.Date.dt.year == 2001].groupby(df.Date.dt.month)['Cancelled'].mean()
ax.bar(x=freq_cancel.index, height=freq_cancel.values, width=1.0);
ax.set_xticks(np.arange(1,13))
ax.set_xticklabels(month_labels);
yticks = mtick.PercentFormatter(xmax=1, decimals=1)
ax.yaxis.set_major_formatter(yticks)
plt.ylabel('Frequency of cancellations');
```

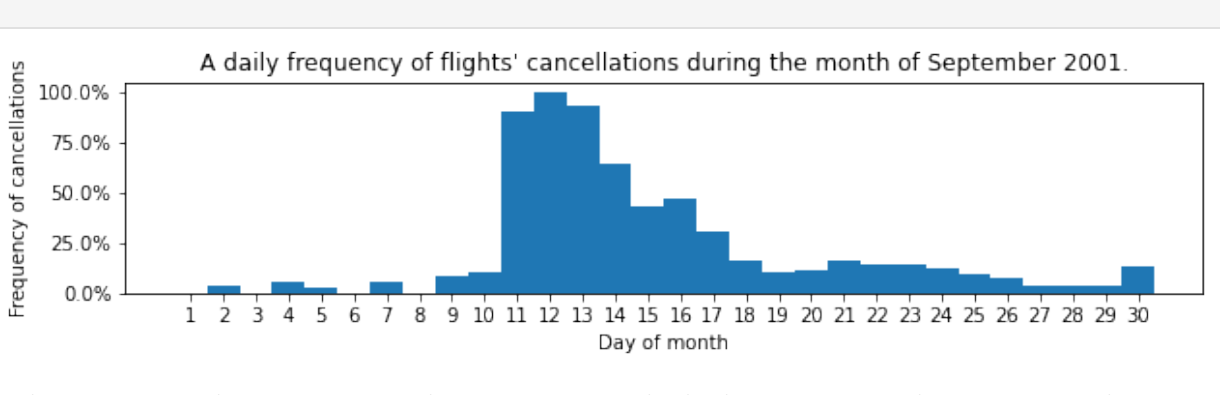


As expected there was probably the highest number of cancellations in history for US domestic flights. Still the percentage of ~20% is probably too low caused by monthly averaging.

To confirm that let's have a look at a daily frequency of flights' cancellations during the month of September 2001 as 22% cancellation rate is calculated as an average for the whole month.

In [31]:

```
# plot daily frequency of cancellations during September 11 (2001)
df_sep11 = df.query('Date.dt.month == 9 and Date.dt.year == 2001')
fig, ax = plt.subplots(1, 1, figsize=(10,2))
freq_cancel = df_sep11.groupby(df_sep11.Date.dt.day)['Cancelled'].mean()
ax.bar(x=freq_cancel.index, height=freq_cancel.values, width=1.0);
ax.set_xticks(np.arange(1,31))
yticks = mtick.PercentFormatter(xmax=1, decimals=1)
ax.yaxis.set_major_formatter(yticks)
ax.set_title("A daily frequency of flights' cancellations during the month of September 2001.")
ax.set_xlabel('Day of month')
ax.set_ylabel('Frequency of cancellations');
```



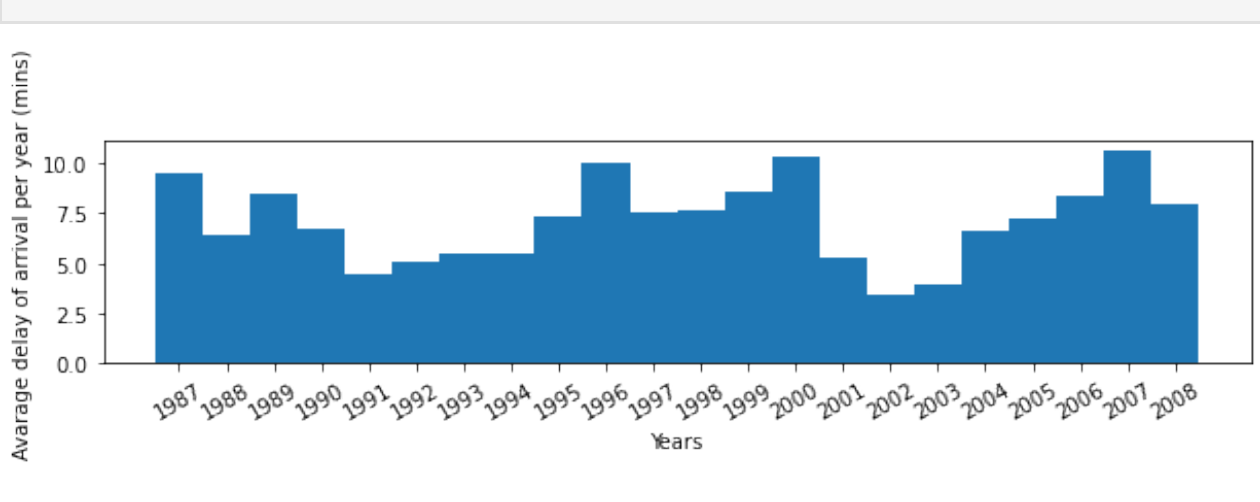
This barchar confirms my assumption that at one point in time the proportion of cancellation must have been close to 100% for the domestic flights in the US.

Question 8:

Is there any trend in departure delays during the years?

In [32]:

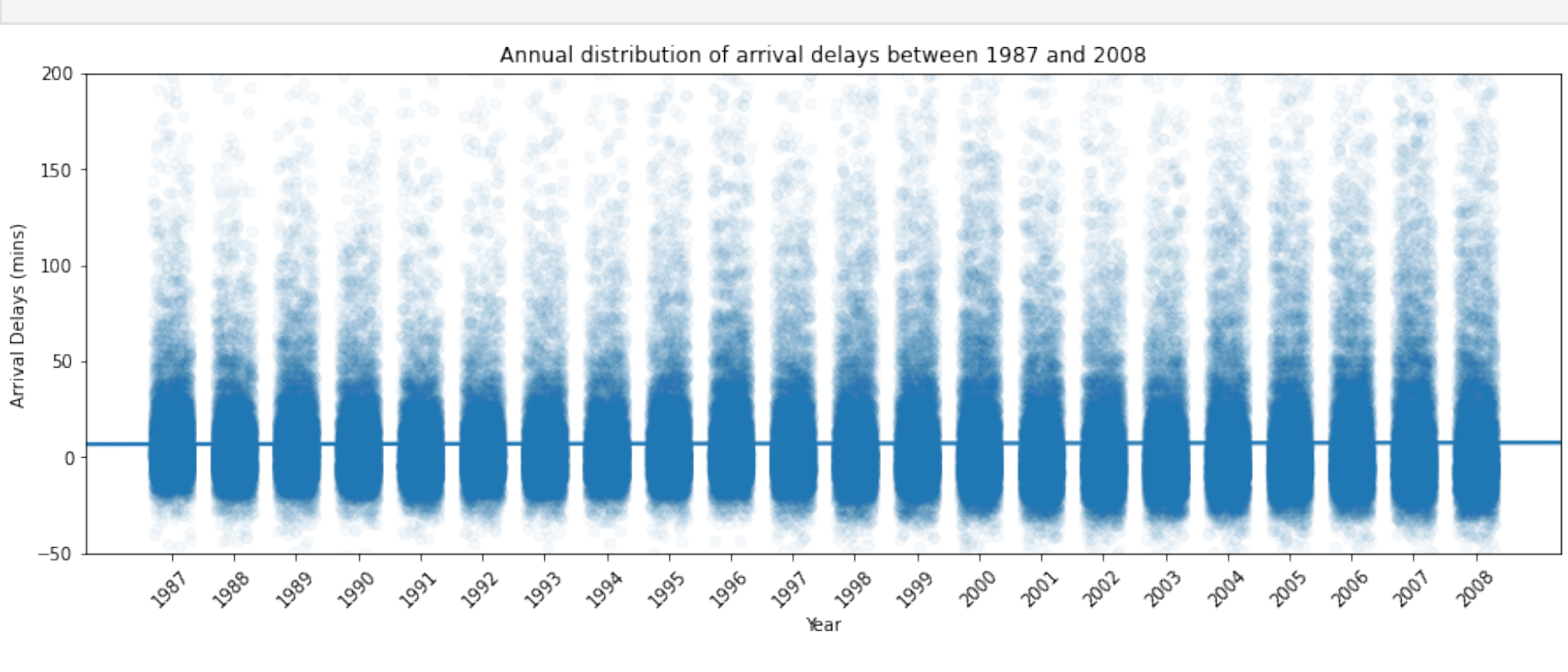
```python
# plot a frequency of arrival delays per year
df_delay_year = df.groupby(df.Date.dt.year)['ArrDelay'].mean().reset_index(name='ArrFreq')
plt.subplots(figsize=(10,2))
plt.bar(data=df_delay_year, x='Date', height='ArrFreq', width=1.0)
plt.xticks(np.arange(1987, 2009), rotation='30')
plt.xlabel('Years')
plt.ylabel('Avarage delay of arrival per year (mins)');
```



The bar chart above shows the average arrival delays during the years. The plot below is showing the distributions of the delays for each year.

In [33]:

```python
# create a new subset of the dataframe and retype delays to avoid issues with seaborn
df_year = pd.DataFrame(df[df['ArrDelay'].isnull() == False][['Date','ArrDelay']])
df_year['Year'] = df_year['Date'].dt.year
df_year['ArrDelay'] = df_year['ArrDelay'].astype(int)

# plot the distribution of arrival delays per year
plt.figure(figsize=[15, 5])
sb.regplot(data=df_year, x='Year', y='ArrDelay', truncate=False, x_jitter=0.3, scatter_kws={'alpha':1/30});
plt.ylim(-50, 200); plt.ylabel('Arrival Delays (mins)')
plt.xticks(np.arange(df_year.Year.min(), df_year.Year.max()+1, 1), rotation=45);
plt.title('Annual distribution of arrival delays between 1987 and 2008');
```



On the first chart showing the average delays there are interesting peaks in 1996, 2000 and 2007, always followed by a significant decrease in average arrival delays per flight, like in 2002. This can be attributed to the strong changes in flights demand during turbulent years of Y2K bubble, 9/11 and GFC. Less flights means less congested airports and less operational delays (take-off clearance, etc). Also, in 1996 there were Olympics in Atlanta obviously causing more disruptions and delays.

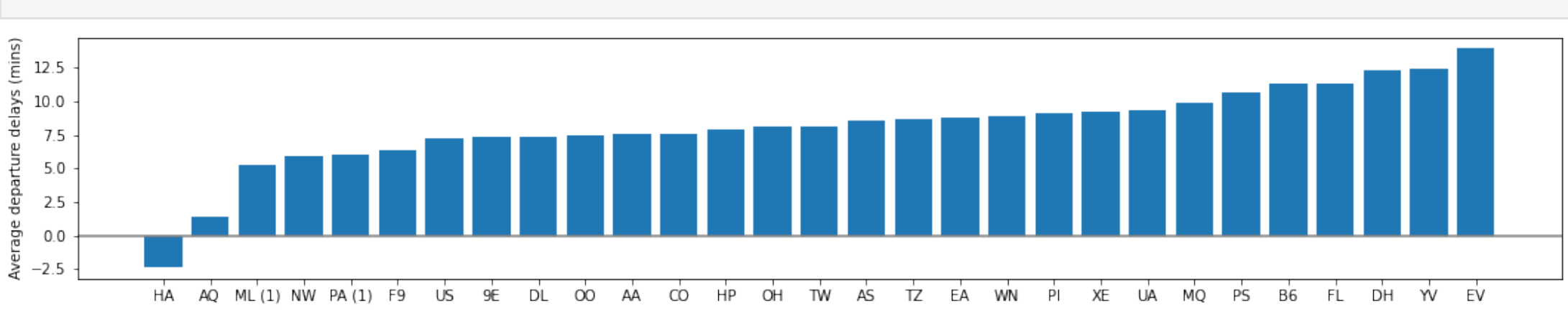
Question 9:

What are the best performing airlines in terms of departure delays?

Let's use a bar chart sorted by average departure delays.

In [34]:

```python
# Sort airlines by departure delays
car_delays = df.groupby('UniqueCarrier')['DepDelay'].mean().sort_values(ascending=True)
plt.subplots(figsize=(18,3))
plt.bar(x=car_delays.index, height=car_delays.values);
plt.axhline(y=0, color='grey')
plt.ylabel('Average departure delays (mins)');
```



Obviously, Hawaiian (HA) is the only carrier with average actual departure times being earlier than planned. Let's investigate HA's performance in more detail in the next section.

Discuss the distribution(s) of your variable(s) of interest. Were there any unusual points? Did you need to perform any transformations?

The exploratory visualization done in the above cells helped to understand the distribution of delays and other variables across all the airlines, airports and for different timeframes. Some of the plots confirmed what we expected but some of them would be good explore further using the bivariate visualization techniques, for example the high number of early departures by HA flights.

Of the features you investigated, were there any unusual distributions? Did you perform any operations on the data to tidy, adjust, or change the form of the data? If so, why did you do this?

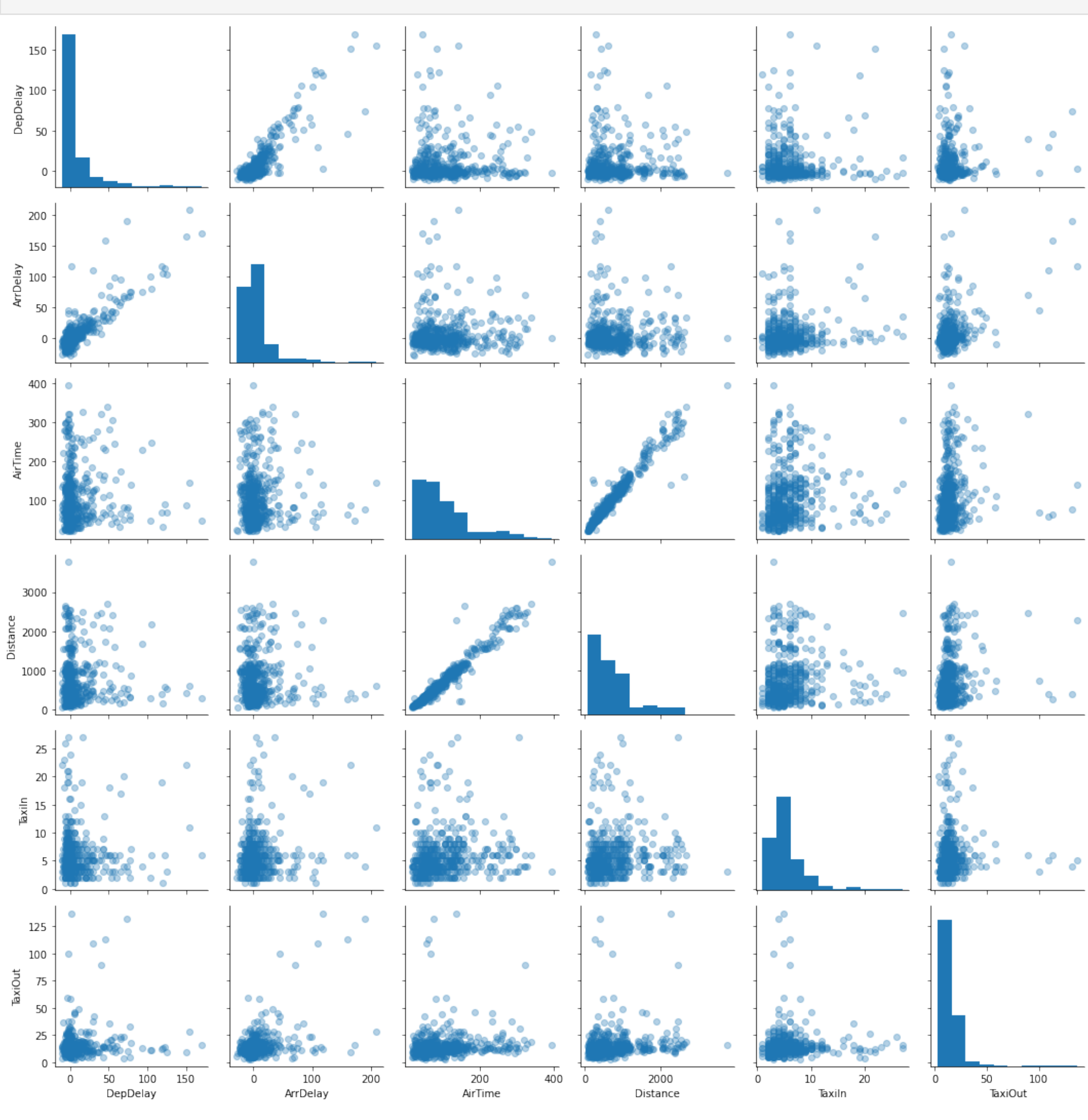
The high occurrence of cancellation in September was unusual but already explained and analyzed in **Question 7**.

Bivariate Exploration

In the previous section we investigated departure and arrival delays as separate variables. Are there any relation between them or other variables? For a quick high-level overlook of the selected qualitative variables I can use a pair grid showing a scatter plot for each pair.

In [35]:

```python
# select numerical qualitative variables
mx_vars = ['DepDelay','ArrDelay','AirTime','Distance','TaxiIn','TaxiOut']

# take only a sample from the original dataset to speed up plotting
# filter some values: don't count cancelled flights or those with negative airtime
df_samp = df.sample(n=800, replace=False).query('Cancelled == 0 and AirTime > 0')

# plot a pair grid of scatter plots
g = sb.PairGrid(data=df_samp[mx_vars], vars=mx_vars)
g.map_diag(plt.hist)
g.map_offdiag(plt.scatter, alpha=1/3);
```



Some of the correlations are as expected, like Distance and Airtime. The other variables seems to be very weekly correlated or not at all except the departure and arrival delays which show a strong linear correlation. This can be expected but let's have a look at this relation in more detail.

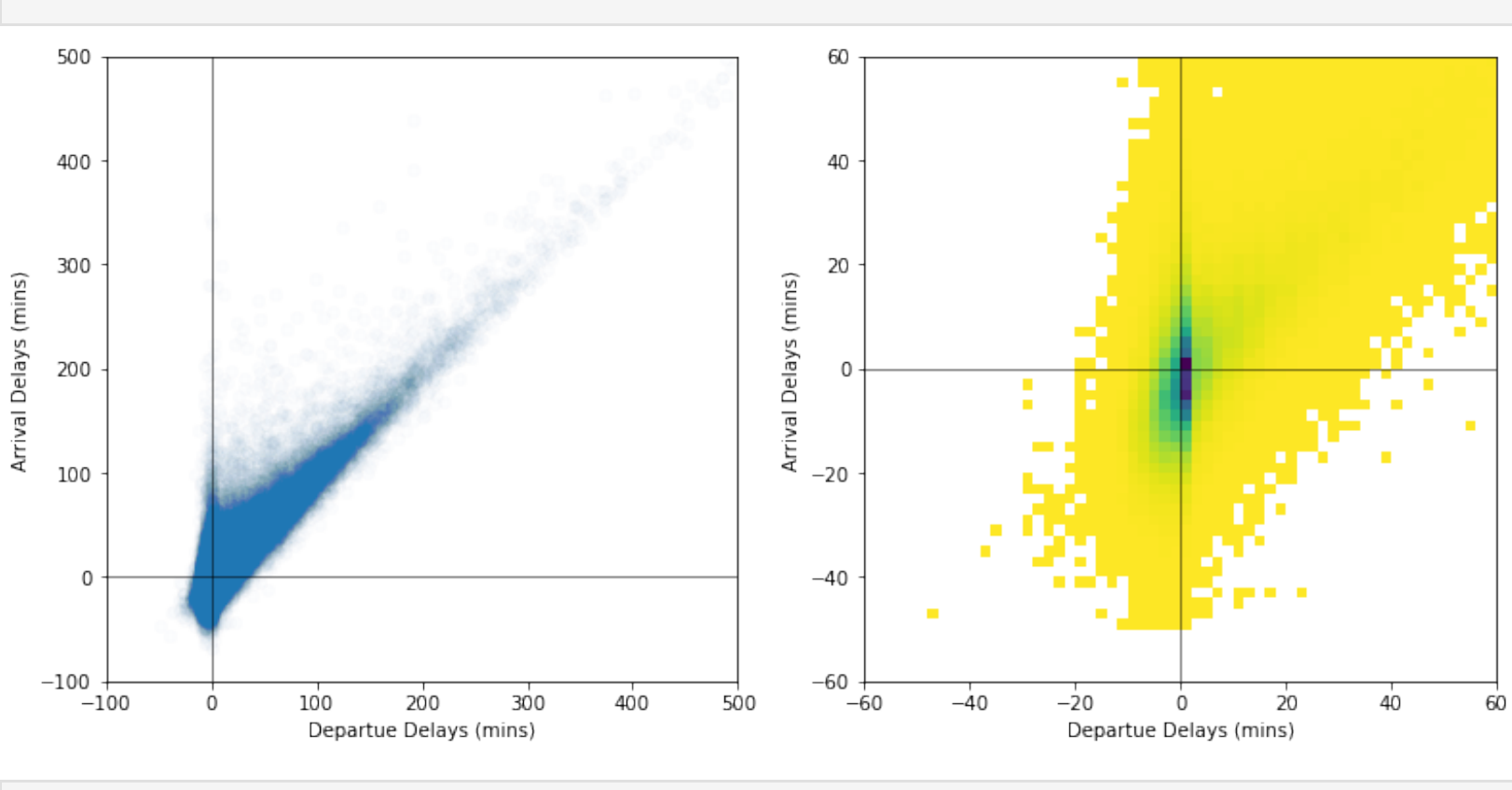
Question 10:

Is there any relation between departure and arrival delays?

Let's use `DepDelay` and `ArrDelay` columns and create a scatter plot.

In [36]:

```python
# plot relation between departure and arrival delays
fig, ax = plt.subplots(1, 3, figsize=(13,6))
# take only those rows where there is some value
df_dep_arr = df.query('not DepDelay.isnull() and not ArrDelay.isnull()')

plt.subplot(1, 2, 1)
plt.scatter(data=df_dep_arr, x='DepDelay', y='ArrDelay', alpha=1/100)
plt.xlim(-100, 500)
plt.ylim(-100, 500)
plt.axvline(0, linewidth=0.5, color='black')
plt.axhline(0, linewidth=0.5, color='black')
plt.xlabel('Departue Delays (mins)')
plt.ylabel('Arrival Delays (mins)');

plt.subplot(1, 2, 2)
bins = np.arange(-50, 500, 2)
plt.hist2d(data=df_dep_arr, x='DepDelay', y='ArrDelay', bins=[bins, bins], cmap='viridis_r', cmin=0.1, cmax=7000)
#plt.scatter(data=df_dep_arr, x='DepDelay', y='ArrDelay', alpha=1/100)
plt.xlim(-60, 60)
plt.ylim(-60, 60)
#plt.colorbar()
plt.axvline(0, linewidth=0.5, color='black')
plt.axhline(0, linewidth=0.5, color='black')
plt.xlabel('Departue Delays (mins)')
plt.ylabel('Arrival Delays (mins)');
```



In [ ]:

The scatterplot on the left is an overall distribution capturing the most of the values. As expected it is showing a strong positive correlation of both variables. The 2D histogram on the right is showing the 'magnified' area around zero values. Most of the flights are around "on-time" area, however there are quite a lot of flights being delayed on departure but arriving on time. It would be interesting to repeat this plot in multivariate section adding another variable, like `AirTime`. It may provide some insight whether the longer airtime allows for making up the departure delays and how much it is utilized by the airlines.

Question 11:

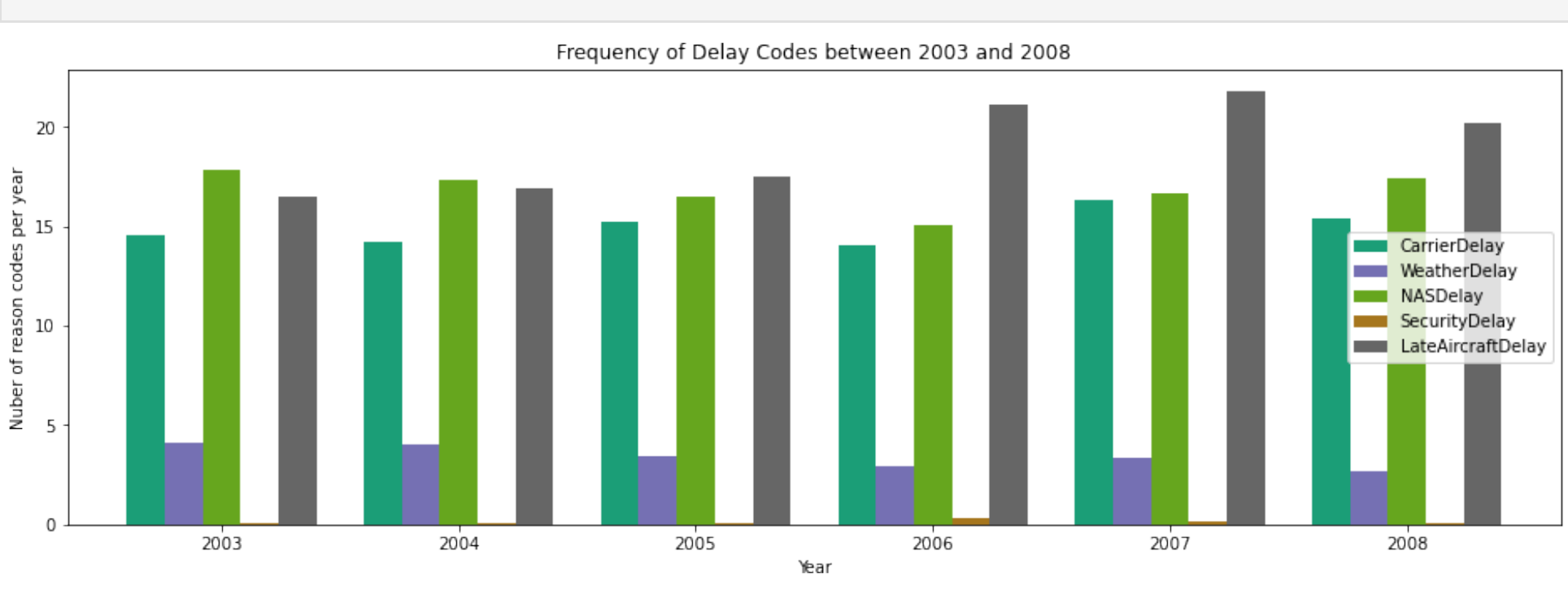
So far we haven't investigated the delay codes distribution. As a part of the bivariate exploration section we can plot to visuialize their distributions across all available years.

What are the proportions of various delay codes and are there any trends during the years?

I will use two ways of visualisations: (1) simple stacked bar chart for the whole dataset, and (2) a seaborn heatmap.

In [37]:

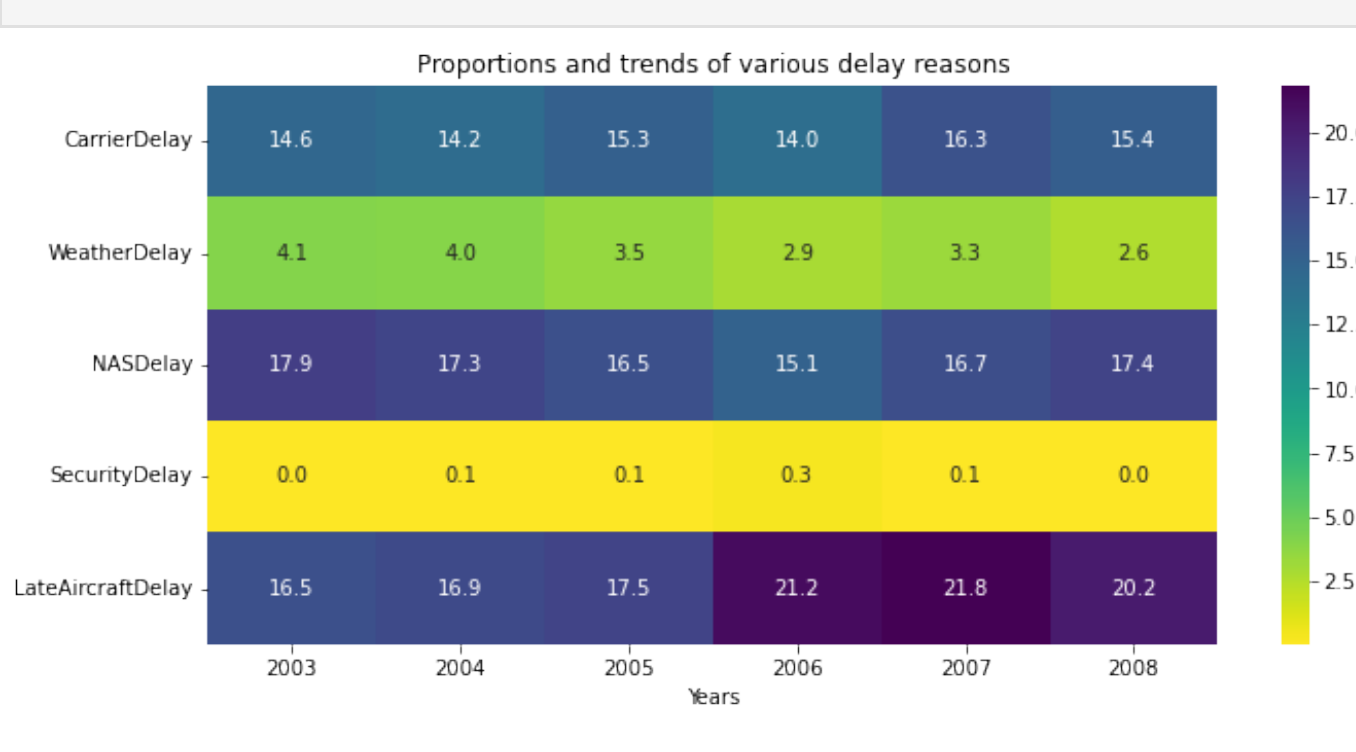
```python
# prepare a new dataframe with relevant columns and rows
df_delays = df.loc[df.Date.dt.year >= 2003][['Date','CarrierDelay','WeatherDelay','NASDelay','SecurityDelay','LateAircraftDelay']]
df_delays = df_delays[df_delays.sum(axis=1) > 0]
delay_mins = df_delays.groupby(df_delays.Date.dt.year).mean()

# plot a stacked barchart per delay reasons; they were first available in 2003
delay_mins.plot.bar(stacked=False, width=0.8, cmap='Dark2', figsize=(16,5))
plt.xticks(rotation=0)
plt.legend(loc='center right')   # don't ovelap bars' edges
plt.title('Frequency of Delay Codes between 2003 and 2008')
plt.xlabel("Year");
plt.ylabel("Nuber of reason codes per year");
```



There are just a few security delays causing visual gaps between the bars, however the trend in delays caused by weather and late aircrafts is quite obvoius. Let's display the same data using a seaborn heatmap.

In [38]:

```python
# this is a workaround for a bug in seaborn.heatmap causing TypeError
df_dmins = pd.DataFrame(delay_mins, dtype='float')

# plot a heatmap for delay reasons
plt.subplots(figsize=(11,5))
# include values inside the cells
sb.heatmap(df_dmins.T, cmap='viridis_r', annot=True, fmt='.1f')
plt.xlabel('Years')
plt.title('Proportions and trends of various delay reasons');
```



The heatmap above depicts average annual delays in minutes across the five categories. The color encoding helps to determine both proportions for each category and possible trends. Obviously, the delays due to weather seems to be trending down while the delays caused by the late connecting flights are growing since 2003.

Question 12:

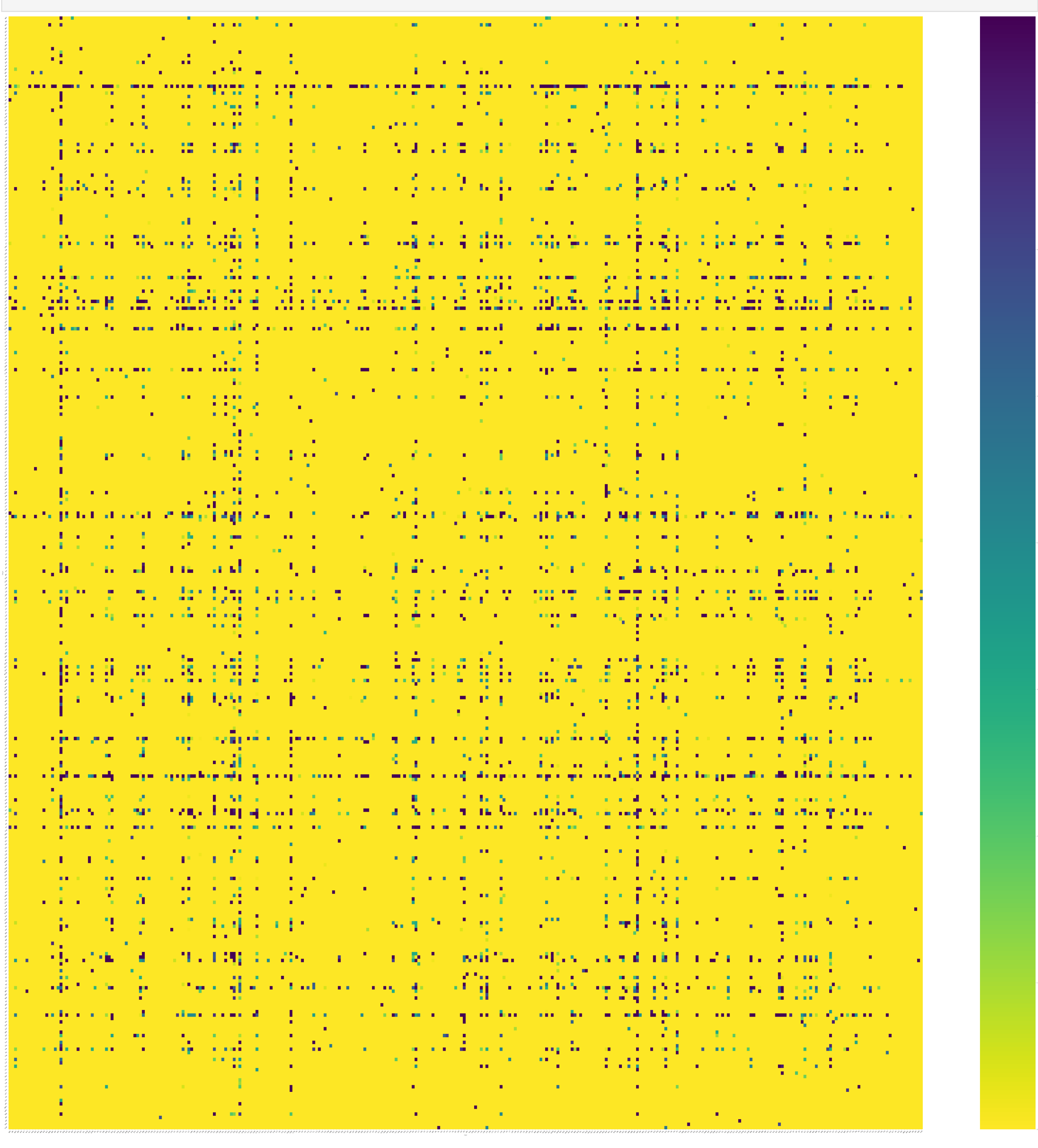
Can we see any patterns in arrival delays distribution across various routes (origin and destination)?

The next heatmap is added here just for curiosity as an 'experimental' exploratory analysis. The plot is huge and only when downloaded and zoomed into the actual size it can provide the intended information. Its cells represent average arrival delays for each origin - destination pair (for all airports in the dataset).

> Please download the heatmap image first and display it in the actual size to see x/y ticks and labels

In [39]:

```python
# plot relation between origin and destination and its delays
# prepare the dataset first
orig_dest_del = df.groupby(['Origin','Dest'])['ArrDelay'].mean()
df_mx = orig_dest_del.reset_index(name='Delay')
df_mx = df_mx.pivot(index='Origin', columns='Dest', values='Delay')
df_mx = pd.DataFrame(df_mx, dtype='float')

# use sb.heatmap (the chart is huge - it is to be downloaded first)
plt.subplots(figsize=(150,150))
sb.heatmap(df_mx.fillna(0), cmap='viridis_r', robust=True, square=False)
plt.yticks(rotation=45); plt.xticks(rotation=45);
```



There are clearly visible patterns in the form of horizontal and vertical lines. This suggest the arrival delays are predominantly airport related. For example Atlanta (ATL) has high delay averages when being origin (on y-axis) or destination (on x-axis) airport.

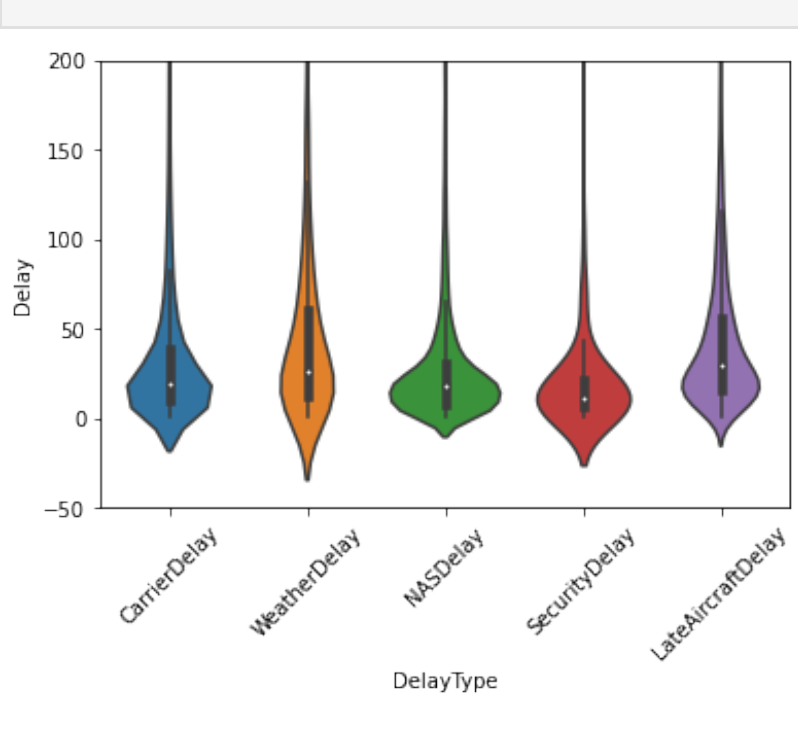
Question 13:

What are the average delays caused by the various reasons?

For the next visualisation I use a violin plot to see how the various delays' are distributed across the delay reasons. It is important to mention that this plot in not showing any information about proportion of each reason type within the whole data. For example there are only a few security caused delays comparing it to weather-related delays.

In [40]:

```python
# created a new dataframe by unpivoting all the delay reasons
df_drea = df[['Date','UniqueCarrier','FlightNum','Distance','CarrierDelay','WeatherDelay','NASDelay','SecurityDelay','LateAircraftDelay']]
df_drea = pd.melt(df_drea.query('Date.dt.year >= 2003'), id_vars=['Date','UniqueCarrier','FlightNum','Distance'], value_vars=['CarrierDelay','WeatherDelay','NASDelay','Secu

# take only non-zero delays
df_drea = df_drea.dropna(axis=0).query('Delay > 0')
df_drea['Delay'] = df_drea['Delay'].astype(int)
#df_drea.describe()

# use a violin plot for the visualisation
sb.violinplot(data=df_drea, x='DelayType', y='Delay')
plt.ylim(-50,200)
plt.xticks(rotation=45);
```



The plot above can be interpreted:

For the delays due to the weather or a late aircraft, if they happen, there is a higher chance they will last a bit longer than the delays for the other reasons.

Talk about some of the relationships you observed in this part of the investigation. How did the feature(s) of interest vary with other features in the dataset?

The scatter plot between departure and arrival delays revealed an interesting deployment of values around zero and negative areas, e.g. earlier than scheduled. It may be further revealing to encode another variable into the plot, like `AirTime`.

It may also be valuable to analyse delays for the specific airports.

Did you observe any interesting relationships between the other features (not the main feature(s) of interest)?

There are trends in the delay reasons between 2003 and 2008. The weather-related delays are slightly on decline while the delays due to the late connecting flights are on the rise. This can be attributed to a continuous evolution of technology helping airlines to manage weather fluctuations. On the other hand increased air traffic and more stringent cost management is introducing more complexity in managing dependencies between connecting flights. But this is just an hypothesis that can explain these two trends.

Multivariate Exploration

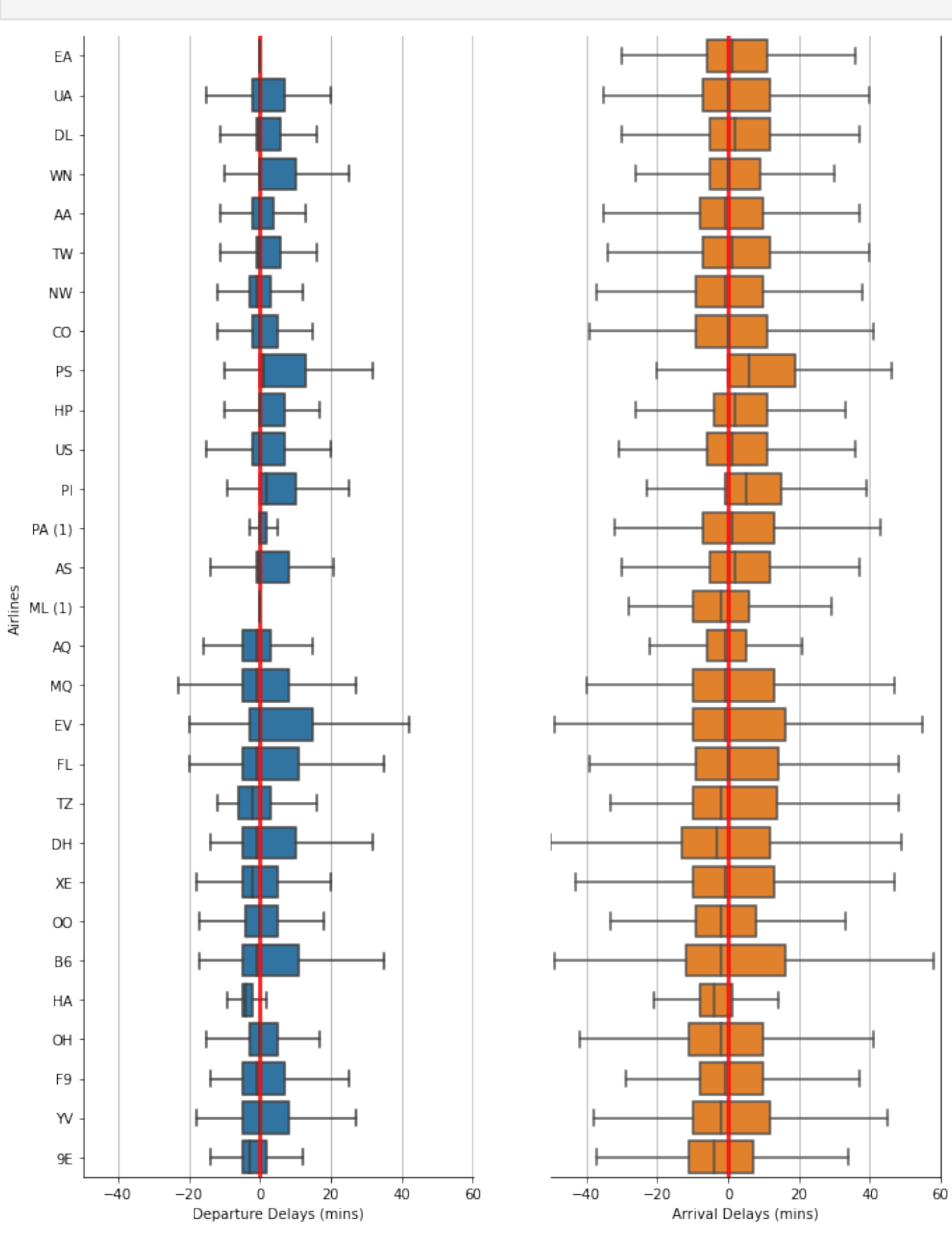
In this section I explored relations among additional variables. Coming back the distribution of departure and arrival delays from the previous section I would like to see its characteristics across all airlines in the dataset. The best plot type seems to be a boxplot. As I noticed some unlikely values (unusually high delays) in the columns I will filter out the outliers for all the plots.

Question 14:

What are the departure and arrival delays distributions across all carriers?

In [41]:

```python
# boxplot for airlines' departure and arrival delays (without outliers)
fig, ax = plt.subplots(1,2,figsize=(11,15), sharey=False)

ax1 = plt.subplot(1,2,1)
base_color = sb.color_palette()[0]
sb.boxplot(data=df, y='UniqueCarrier', x='DepDelay', showfliers=False, color=base_color)
plt.axvline(0, linewidth=3, color='red', alpha=0.8);
plt.xlabel('Departure Delays (mins)'); plt.ylabel('Airlines')
ax1.spines[['right','top']].set_visible(False)
ax1.grid(axis='x')
plt.xlim(-50,60.1)

ax2 = plt.subplot(1,2,2)
base_color = sb.color_palette()[1]
sb.boxplot(data=df, y='UniqueCarrier', x='ArrDelay', showfliers=False, color=base_color)
plt.axvline(0, linewidth=3, color='red', alpha=0.8);
plt.xlabel('Arrival Delays (mins)'); plt.ylabel('')
ax2.set_yticks([])
ax2.spines[['left','top']].set_visible(False)
ax2.grid(axis='x')
plt.xlim(-50,60.1);
```



The ranges of x-ticks are the same for both departures and arrivals to facilitate the comparison. As expected the arrival delays also have to 'absorb' the departure delays so the spreads are much higher.

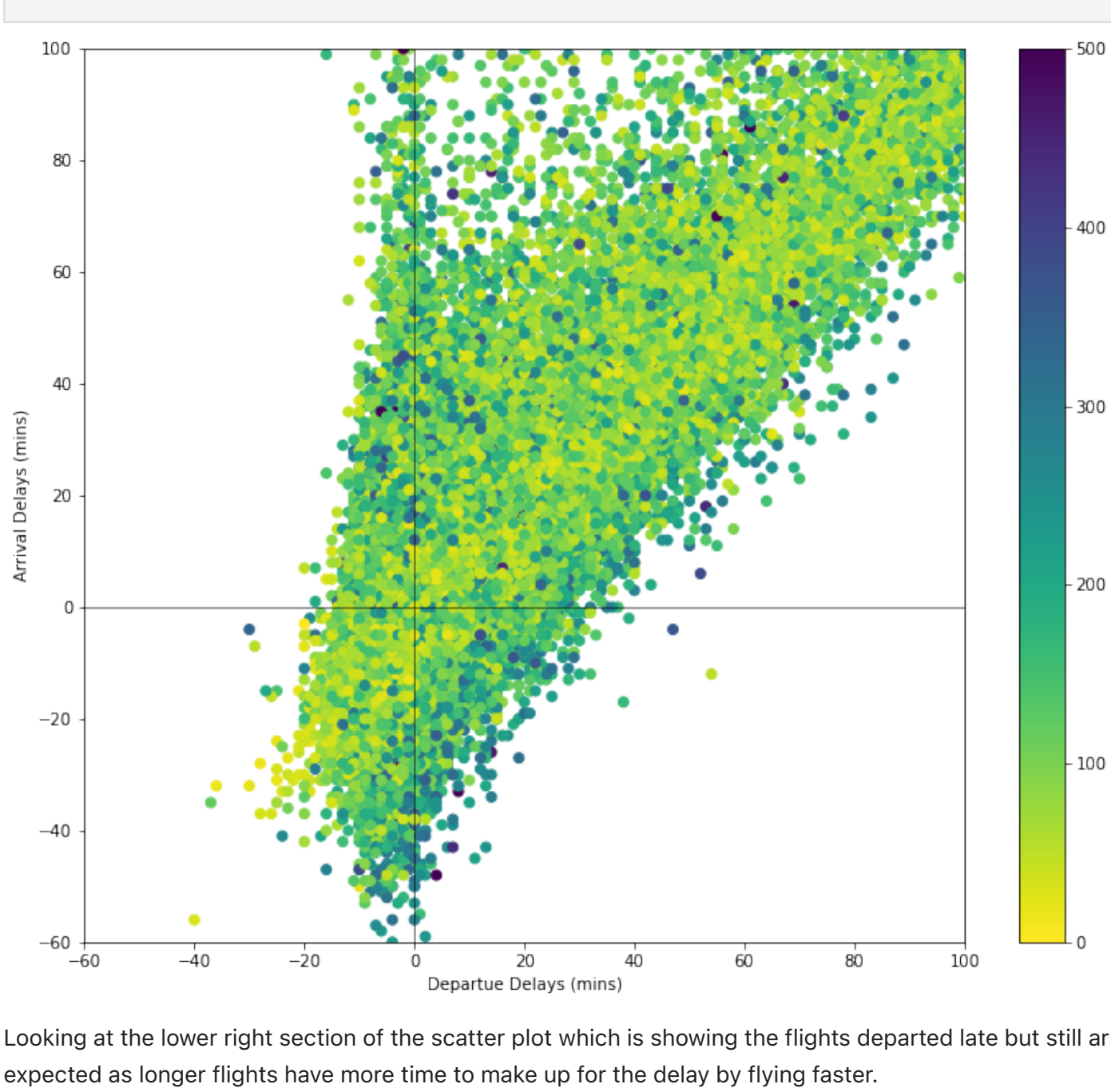
Question 15:

Is there any relation between departure/arrival delays and the airtime?

This visualization is based on Question 10 scatter plot. It is adding `AirTime` as a new variable using color encoding. The intention is to see whether there is any pattern in the distribution, especially for the areas with negative delays (early departure or arrival).

In [42]:

```python
# plot relation between departure and arrival delays
fig, ax = plt.subplots(figsize=(12, 10))
# take only those rows where there is some value
df_dep_arr = df.query('not DepDelay.isnull() and not ArrDelay.isnull()')

plt.scatter(data=df_dep_arr, x='DepDelay', y='ArrDelay', c='AirTime', cmap='viridis_r', alpha=1)
plt.xlim(-60, 100)
plt.ylim(-60, 100)
plt.clim(0, 500)
plt.colorbar()
plt.axvline(0, linewidth=0.5, color='black')
plt.axhline(0, linewidth=0.5, color='black')
plt.xlabel('Departue Delays (mins)')
plt.ylabel('Arrival Delays (mins)');
```



Looking at the lower right section of the scatter plot which is showing the flights departed late but still arriving earlier than schedulled, we can see a higher frequency of longer flights (dark/blue). This is as expected as longer flights have more time to make up for the delay by flying faster.

Talk about some of the relationships you observed in this part of the investigation. Were there features that strengthened each other in terms of looking at your feature(s) of interest?

Were there any interesting or surprising interactions between features?

The first plot in the Multivariate section showed each airline's on-time performance for both departures and arrivals. There was one airline that managed to have the most of the departures and arrivals earlier than scheduled - HA Hawaiian Airlines. I tried to find some supporting evidence for the assumption that HA are the best in on-time performance and found this article from Feb 25, 2021:

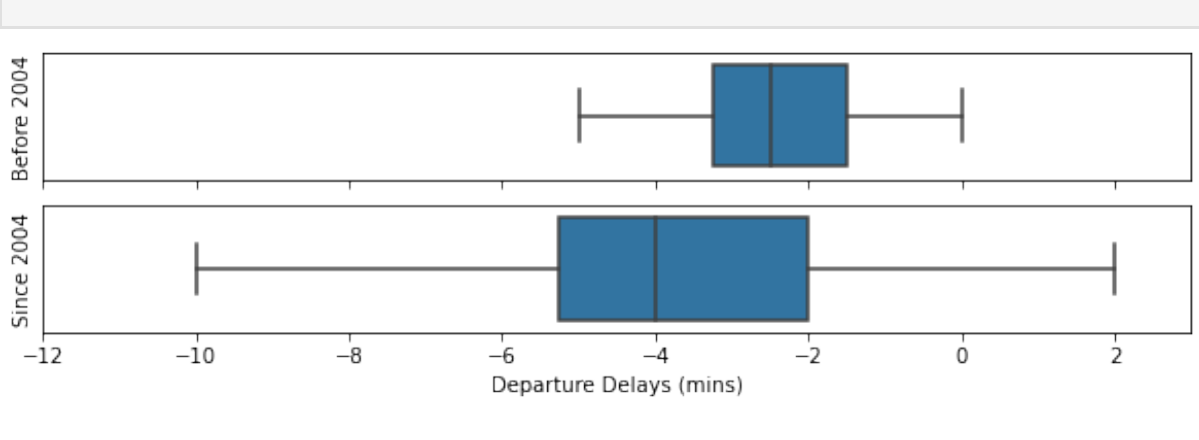
https://newsroom.hawaiianairlines.com/releases/hawaiian-airlines-maintains-on-time-performance-record-for-17th-consecutive-year

It mentioned that *Hawaiian have began its on-time performance streak in 2004*. Let's compare Hawaiian's performance before and since 2004.

> The following visualizations depict single variables and thus they don't belong to this multivariate section. They are just to support the previous conclusion.

In [43]:

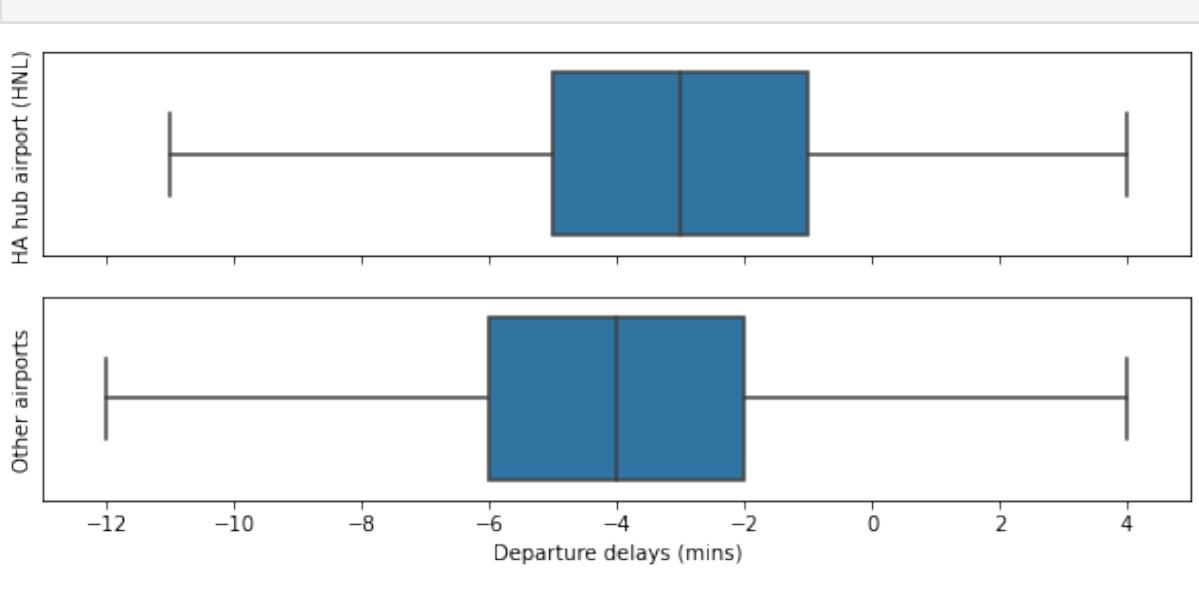
```python
# comparison boxplots for HA - before and since 2004
plt.subplots(2, 1, figsize=(10, 2.5), sharex=True)

plt.subplot(2,1,1)
sb.boxplot(data=df.query('UniqueCarrier == "HA" and Date.dt.year < 2004'), x='DepDelay', showfliers=False)
plt.xlim(-12,3)
plt.xlabel(''); plt.yticks([])
plt.ylabel('Before 2004')

plt.subplot(2,1,2)
sb.boxplot(data=df.query('UniqueCarrier == "HA" and Date.dt.year >= 2004'), x='DepDelay', showfliers=False)
plt.xlim(-12,3);
plt.xlabel('Departure Delays (mins)')
plt.ylabel('Since 2004'); plt.yticks([]);
```



There is a significant improvement in departure times since 2004. As a final step let's have a look if the departure delays performance is different between Hawaiian hub airport at Honolulu (HNL) and the other airports. By that we would see if the early departures were caused mainly by the advantage of the 'home' airport or if such a good performance is across all ground-handled operations for HA.

In [44]:

```python
# boxplot for HNL and 'other' airports showing HA's departure delays
plt.subplots(2, 1, figsize=(10, 4), sharex=True)

plt.subplot(2, 1, 1)
sb.boxplot(data=df.query('Origin=="HNL" and UniqueCarrier=="HA" and Date.dt.year >= 2004'), x='DepDelay', showfliers=False)
plt.xlim(-13, 5)
plt.ylabel('HA hub airport (HNL)')
plt.xlabel(''); plt.yticks([])

plt.subplot(2, 1, 2)
sb.boxplot(data=df.query('Origin!="HNL" and UniqueCarrier=="HA" and Date.dt.year >= 2004'), x='DepDelay', showfliers=False)
plt.xlim(-13, 5); plt.yticks([]);
plt.ylabel('Other airports')
plt.xlabel('Departure delays (mins)');
```



There seems to be no significan difference so the good performance is not caused just by the home environment at HNL airport.

Conclusion

The aim of this jupyter notebook was to analyse airline's on-time performance dataset using various visualization techniques. This first section focused on preliminary data analysis and data wrangling. The next univariate section investigated the distribution of the main variables of interest and revealed some unusual distribution of flight cancellations per month which was later explained as being attributed to the events of September 11th. The next two sections investigated further the relation between the departure and arrival delays and other variables using bivariate and multivariate visualizations. The last part of the notebook focused on the explanation of Hawaiian airlines above average occurrence of early departures and arrivals.